

Ministry of Education and Science of the Republic of Kazakhstan
Astana IT University

REPORT
about the implementation of the internship program

Practice base: TOO «PetroSkills Pro»
Student: Mukhtarova Kamilla
Educational program: Software Engineering
Group: SE-2309

Enterprise practice supervisor: Askhat Kuzhakhmetov
University practice supervisor: Elvira Aitmukhanbetova

Astana 2026

Contents

1. Introduction	3
1.1. Place of Practice	3
1.2. Goals and Objectives	3
1.3. Expected Results of Industrial Practice	4
2. Main Part	4
2.1. Theoretical Part	4
2.1.1. General Information About the Company	4
2.1.2. Organizational Structure of the Department	4
2.2. Practical Part	4
2.2.1. Project Overview: Sana LMS Platform	5
2.2.2. Research and Analysis of AI-Chat Architecture	5
2.2.3. Design of AI Assistant for LMS	5
2.2.4. Implementation of AI Assistant (MVP)	5
2.2.5. Integration with Sana Platform	6
2.2.6. Technologies Used	6
2.2.7. Challenges and Solutions	6
3. Conclusion	7
4. Reflection	7
5. References	8
6. Practice Task	8
7. Appendices	9

1. Introduction

Industrial practice is an essential part of the educational process aimed at developing practical skills in system analysis, backend development, and AI integration within a real-world project environment. This report presents the results of the industrial practice completed as part of the Software Engineering educational program.

The practice was focused on participating in the development of a modern Learning Management System (LMS) called Sana, which is designed for corporate training and employee development. The platform integrates modules such as course management, training requests, certification tracking, and learning analytics, providing a complete ecosystem for employee development.

During the internship, special attention was given to the design and development of an AI assistant based on Large Language Models (LLM), integrated into the LMS platform. The assistant is intended to support users by providing guidance, answering questions, and simplifying interaction with the system.

1.1. Place of Practice

The industrial practice was completed remotely in collaboration with PetroSkills Pro LLP, which is involved in the development of a startup project - AI-LMS, an intelligent learning management system for corporate training.

The project is developed in cooperation with the company Nomad Speakers, which specializes in organizing corporate training, business events, and implementing digital learning solutions in Kazakhstan.

The Sana platform represents a comprehensive LMS system designed to support corporate education processes, improve employee competencies, and provide scalable and flexible learning solutions using modern technologies.

1.2. Goals and Objectives

The main goal of the industrial practice was to gain practical experience in software development, with a focus on backend systems, system architecture, and AI integration within a corporate LMS platform.

The key objectives of the practice included:

- studying the architecture and functionality of the Sana LMS platform
- analyzing AI-chat architectures based on the Encore framework
- comparing modern LLM models (Gemini, GPT, Claude) for integration into the system
- designing an AI assistant adapted for LMS use cases
- developing system prompts for user-friendly AI interaction
- participating in the development of backend API endpoints for AI communication
- integrating AI capabilities into the LMS environment
- improving skills in backend and frontend development

- working collaboratively within a development team

1.3. Expected Results of Industrial Practice

As a result of the industrial practice, it was expected to develop both technical and professional competencies in software engineering, including system analysis, architecture design, and AI integration.

The expected outcomes included:

- development of a functional LMS platform prototype
- implementation of an AI assistant integrated into the system
- development of a functional MVP AI assistant capable of generating responses to user queries in real time
- gaining experience in working with modern technologies such as Go (Encore), Next.js, and LLM-based systems
- understanding real-world software development processes and teamwork

The practice also aimed to enhance problem-solving skills, improve technical knowledge, and prepare for future professional activities in the field of software engineering.

2. Main Part

2.1. Theoretical Part

2.1.1. General Information About the Company

PetroSkills Pro LLP is a Kazakhstan-based company developing an AI-powered Learning Management System (AI-LMS) for corporate training. The project is carried out in cooperation with Nomad Speakers, a company that specializes in organizing corporate training events and implementing digital learning solutions across Kazakhstan.

The Sana LMS platform serves as the core product - a comprehensive system for managing corporate education, tracking employee progress, handling certifications, and enabling data-driven learning analytics.

2.1.2. Organizational Structure of the Department

The development team includes frontend and backend engineers, system analysts, and project managers. During the practice, the intern worked within the backend and AI integration sub-team, collaborating closely with the frontend team for full-stack feature delivery. Communication was conducted via standard remote development tools, with regular check-ins and code reviews.

2.2. Practical Part

During the industrial practice, a significant part of the work was focused on the research, design, and implementation of an AI assistant integrated into the Sana LMS platform.

At the initial stage, participation in the MVP development was carried out in the role of a system analyst. Responsibilities included designing user flows, defining interface logic

(notifications, attendance tracking, empty states), and reviewing frontend code for security aspects such as proper use of environment variables. Additionally, contributions were made to documentation and intermediate presentations were prepared to support the development process.

2.2.1. Project Overview: Sana LMS Platform

Sana LMS is a modern corporate learning management system built to support employee training and development. The platform includes the following key modules: course management, training requests, certification tracking, employee progress analytics, notifications, and external learning (SCORM). The system supports multiple user roles - employee, HR manager, and administrator - each with distinct access rights and functionality.

2.2.2. Research and Analysis of AI-Chat Architecture

Further work was focused on analyzing the possibility of integrating an AI assistant into the LMS platform. The main objective of the assistant is to support users by answering questions, explaining system functionality, and providing personalized recommendations for learning.

The architecture of existing AI-based systems was analyzed, including the ai-chat project built on the Encore framework. Special attention was given to understanding the interaction between components such as chat orchestration, LLM services, and provider layers.

Additionally, a comparative analysis of modern Large Language Models (LLMs) was conducted, including Claude, GPT-4o, and Gemini. The evaluation was based on key criteria such as response quality, speed, cost, and the ability to handle business logic scenarios.

As a result of this analysis, Gemini was selected as the primary model for MVP implementation due to its balance between performance and cost efficiency.

2.2.3. Design of AI Assistant for LMS

Based on the research, a simplified architecture of the AI assistant tailored to LMS requirements was designed. The architecture follows a request-response model:

Next.js → /assistant/chat → Backend (Encore, Go) → Gemini → Response

In this architecture:

- the frontend (Next.js) sends user queries to the backend
- the backend processes the request and enriches it with business context (such as user role, course data, and learning progress)
- the request is sent to the Gemini model
- the generated response is returned to the user

A separate assistant service was designed to handle AI-related logic, including prompt processing, context preparation, and communication with the LLM.

2.2.4. Implementation of AI Assistant (MVP)

At the implementation stage, a working MVP of the AI assistant was developed. The backend includes a fully functional API endpoint (/assistant/chat) implemented using Go and the Encore framework. This endpoint processes user input, enriches it with contextual data (such as user role, courses, certificates, and employee information), and sends a structured prompt to the Gemini model.

The AI assistant uses role-based logic, adapting responses depending on the user type (employee, HR, admin), which improves the relevance and usefulness of responses. The integration with the Gemini API was completed, enabling real-time generation of responses based on user queries.

On the frontend side, a chat interface was implemented using Next.js and integrated into the Sana LMS interface. The UI allows users to send messages and receive responses in real time via HTTP requests to the backend.

The developed MVP was successfully tested in a local development environment, where the frontend and backend were run simultaneously. The AI assistant correctly processed user requests and returned generated responses through the integrated chat interface.

2.2.5. Integration with Sana Platform

In addition to the AI assistant, a Python script was developed to automatically parse the Next.js routing structure. This script extracts routes, functions, and imports from the project, which helps the AI assistant better understand the application structure and provide more relevant responses.

This approach improves the contextual awareness of the assistant and supports more accurate answers related to system navigation and functionality. A reusable React component (AiChat) was created using Next.js, including an input field, a send button, and a response display area. It communicates with the backend via HTTP POST requests to the /assistant/chat endpoint.

The chat interface was embedded into an existing LMS page (SCORM module page), ensuring consistency with the overall UI design of the platform.

2.2.6. Technologies Used

During the practice, the following technologies were used:

- Go (Encore) - backend development
- Next.js - frontend development
- Gemini API - integration of AI capabilities
- Python - auxiliary scripts for data processing
- Git / VS Code - development and collaboration tools

2.2.7. Challenges and Solutions

During the development process, several challenges were encountered:

- Understanding AI architecture - initially, it was difficult to understand how LLM-based systems are structured. This was resolved through analysis of existing solutions such as ai-chat.
- Choosing the appropriate LLM model - different models were evaluated to find the optimal balance between performance and cost.
- Working with API integration - difficulties were encountered when configuring and testing API requests, which required additional debugging and experimentation.
- Designing effective prompts - it was necessary to structure prompts in a way that ensures consistent and accurate responses from the AI assistant.

These challenges contributed to gaining deeper practical knowledge and improving problem-solving skills.

3. Conclusion

The industrial practice provided valuable experience in applying theoretical knowledge to a real-world software engineering project. During the internship, the development of a corporate Learning Management System (Sana) was carried out with contributions to both analytical and technical aspects of the project.

One of the key outcomes of the practice was the successful development of a working MVP AI assistant integrated into the Sana LMS platform. The assistant supports real-time interaction with users, processes contextual information from the system, and generates intelligent responses using a Large Language Model (Gemini).

The implementation includes both backend and frontend components, forming a complete end-to-end feature within the LMS. This allowed the assistant to be directly used inside the platform through a chat interface.

In addition, practical experience was gained working with modern technologies such as Go (Encore), Next.js, and LLM-based APIs. Skills in system analysis, architecture design, and collaborative development within a team environment were also strengthened.

Overall, the practice significantly enhanced understanding of building scalable systems and integrating AI into real-world applications. It also improved the ability to solve complex problems, work with new technologies, and adapt to project requirements.

The knowledge and experience gained during this practice will be valuable for future professional development in software engineering, particularly in backend development and AI-driven systems.

4. Reflection

During the industrial practice, valuable experience was gained that significantly expanded understanding of software engineering and real-world project development.

At the beginning of the internship, one of the main challenges was understanding the architecture of a complex system and how its components interact. Working on the Sana LMS project helped develop system thinking and understand how frontend, backend, and AI components are integrated within a single application.

A particularly important part of the experience was working with AI technologies. Initially, practical experience with Large Language Models was limited, but during the practice different models were analyzed, their capabilities were compared, and the most suitable one for the project was selected. System prompts were also designed, and it became clear how prompt structure directly affects the quality and consistency of AI responses.

Another valuable aspect was working with backend-related tasks and external APIs. Involvement included designing the interaction flow of the AI assistant and implementing the backend endpoint for communication with the LLM. Although this process included challenges related to API integration and system architecture, they were overcome through testing, debugging, and continuous learning.

A particularly valuable achievement was the implementation of a full-stack AI feature, including both backend logic and frontend integration. This allowed for a better understanding of how different system layers interact in real-world applications.

Overall, this practice helped to become more confident in working with complex systems and strengthened practical skills in AI integration and backend development. It also helped identify key areas for further growth, particularly in designing scalable systems and working with AI-driven solutions.

This experience confirmed interest in software engineering and motivated continuing development in the field of AI-based applications.

5. References

1. Encore Documentation. Available at: <https://encore.dev/docs>
2. Next.js Documentation. Available at: <https://nextjs.org/docs>
3. Google Gemini API Documentation. Available at: <https://ai.google.dev>
4. Google Cloud Vertex AI Documentation. Available at: <https://cloud.google.com/vertex-ai>
5. OpenAI. Introduction to Large Language Models. Available at: <https://platform.openai.com/docs>
6. Anthropic Claude Documentation. Available at: <https://docs.anthropic.com>
7. Encore AI Chat Example Repository. Available at: <https://github.com/encoredev/examples>

6. Practice Task

During the industrial practice, tasks were assigned related to the development of a Learning Management System (Sana) with a focus on system analysis, backend development, and AI integration.

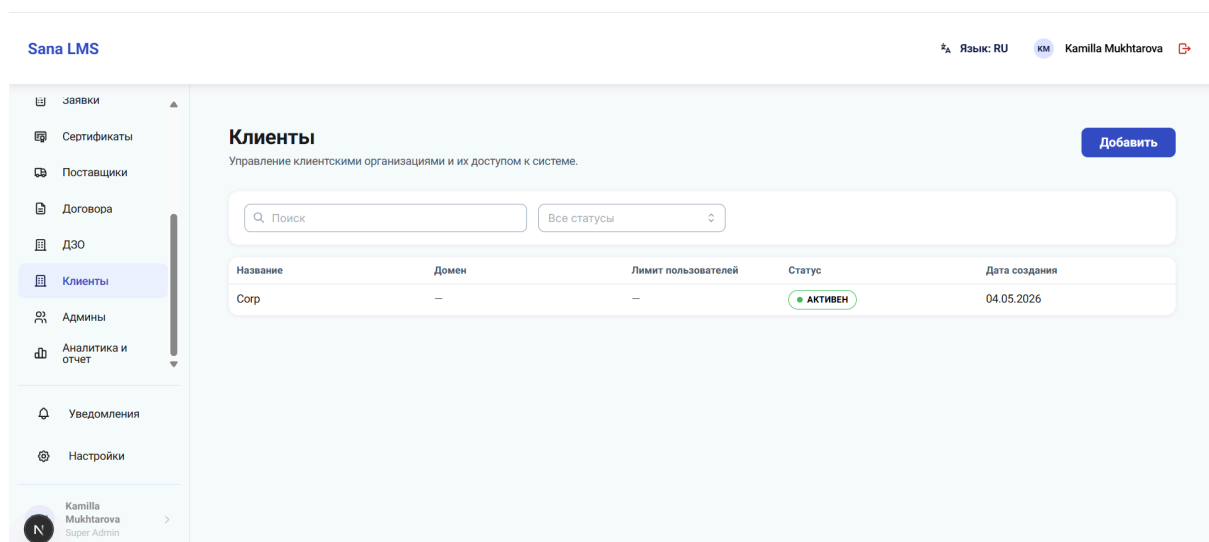
Main responsibilities included:

- analyzing the architecture and functionality of the Sana LMS platform
- participating in MVP development as a system analyst
- designing user flows and interface logic (notifications, attendance tracking, empty states)
- reviewing frontend code for basic security practices (environment variables usage)
- researching AI assistant integration into LMS systems
- analyzing AI-chat architecture based on the Encore framework
- comparing modern Large Language Models (Gemini, GPT, Claude)
- designing the architecture of an AI assistant for LMS
- developing system prompts for AI interaction
- implementing the backend API endpoint (/assistant/chat) for AI communication
- integrating the Gemini API into the backend service
- building context-aware logic using user data (role, courses, certificates, employee info)
- developing a frontend chat component using Next.js and integrating it into the LMS interface
- testing the full interaction flow between frontend, backend, and AI model
- debugging API integration and improving response quality

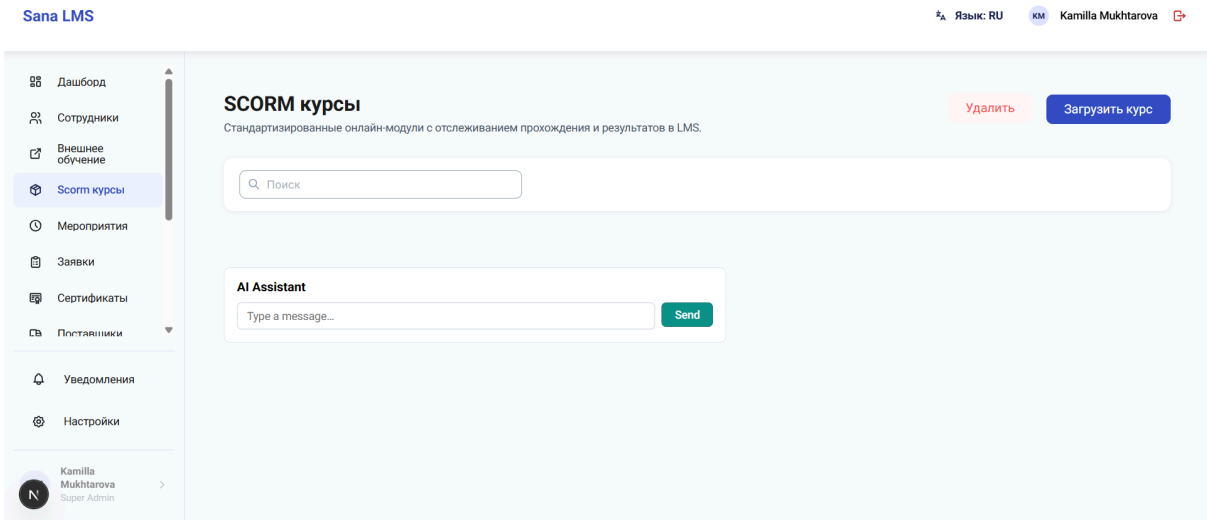
These tasks allowed gaining both analytical and practical experience in software development and AI system design.

7. Appendices

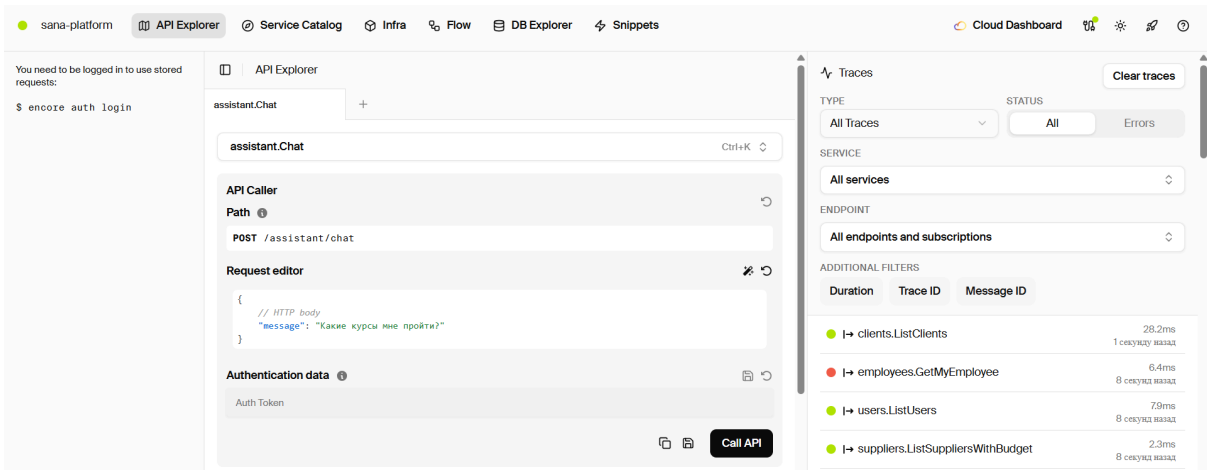
Appendix A - LMS Interface



Appendix B - AI Assistant UI



Appendix C - Backend API Testing



Appendix D - Code Implementation

```
sana-platform > assistant > assistant.go 1 of 2 | 1 of 3 Files | Undo File Keep File Ctrl+g ПРАКТИКА БЭК АИ
1 package assistant
2
3 import (
4     "context"
5     "fmt"
6     "strings"
7
8     "encore.app/auth/authhandler"
9     "encore.app/courses/events"
10    "encore.app/learning/certificates"
11    "encore.app/orgstructure/employees"
12    "encore.dev/beta/auth"
13    "encore.dev/beta/errs"
14 )
15
16 // ===== DATABASE =====
17
18 // DB section intentionally left empty for skeleton stage.
19
20 // ===== ENDPOINTS =====
21
22 var gemini = newGeminiClient()
23
24 // Chat sends a message to the AI assistant and returns its reply.
25 //
26 //encore:api public method=POST path=/assistant/chat
27 func Chat(ctx context.Context, req *ChatRequest) (*ChatResponse, error) {
28     ad, _ := getAuthData()
29
30     userContext := buildContext(ctx, ad, req.Message)
31     role := "anonymous"
32     if ad != nil {
```

```
практика бэк аи Agents Window
assistant_types.go gemini.go M secrets.local.cue
sana-platform > assistant > gemini.go 3 of 5 | 2 of 3 Files | Undo File Keep File Ctrl+g ПРАКТИКА БЭК АИ
25
26 type geminiClient struct {
27     apiKey string
28     endpoint string
29     http *http.Client
30 }
31
32 func newGeminiClient() *geminiClient {
33     return &geminiClient{
34         apiKey: secrets.GeminiAPIKey,
35         endpoint: geminiEndpoint,
36         http: defaultHTTPClient,
37     }
38 }
39
40 // ===== GEMINI REST TYPES =====
41
42 type geminiRequest struct {
43     Contents []geminiContent `json:"contents"`
44 }
45
46 type geminiContent struct {
47     Parts []geminiPart `json:"parts"`
48 }
49
50 type geminiPart struct {
51     Text string `json:"text"`
52 }
53
54 type geminiResponse struct {
55     Candidates []geminiCandidate `json:"candidates"`
56     Error *geminiError `json:"error,omitempty"`
57 }
```